Generating Ergonomic Fingerings for Piano

Mikael Andersson
Mira Yao Håkansson

Kurs: Examensarbete inom datalogi, grundnivå (DD143X)

Kungliga Tekniska högskolan
Skolan för datavetenskap och kommunikation (CSC)

Handledare: Anders Askenfelt

Kontaktinformation

Mikael Andersson	Mira Yao Håkansson
Adress: Ekedalsv. 15D	Adress: Sköndalsvägen 187
177 59 Järfälla	128 68 Sköndal
Tel. nr: 070-446 45 31	Tel. nr: 073-630 38 41
Email: mikaela7@kth.se	Email: mirayh@kth.se

Sammanfattning

Genomtänkta fingersättningar är användbara för att kunna spela ett instrument väl, och bidrar till att förbättra både flexibilitet och ljudkvalitet. Denna studie utforskade möjligheterna att med hjälp av en uppsättning regler generera ergonomiska fingersättningar för piano, för monofoniska melodier. En algoritm baserad på dynamisk programmering implementerades, och trellisgrafer användes för att illustrera de genererade fingersättningarna. Algoritmens prestationsförmåga utvärderades genom att jämföra de givna resultaten med fingersättningar föreslagna av två professionella pianister. Mängden av automatiskt genererade fingersättningar inkluderade stora delar av de fingersättningar som föreslagits av pianisterna, men också en stor mängd fingersättningar som inte höll lika hög kvalitet.

Abstract

In order to play an instrument well, carefully planned fingerings can be helpful and improve flexibility and sound quality. This study explores the possibilities of generating ergonomic piano fingerings for monophonic melodies based on a set of rules. An algorithm is implemented using dynamic programming, and trellis graphs are used to illustrate the possible fingerings. The performance of the algorithm was evaluated by comparing the obtained results with fingerings suggested by two professional pianists. The automatically generated set of fingerings included large parts of the fingerings suggested by the pianists, but also a large superset of other fingerings of lower quality.

Innehållsförteckning

1. Introduction to Musical Terms....1
2.1. Background....2
2.2. Problem Statement....3
3. Method....3
3.1. Approach....3
3.2. Mathematical Definitions....5
3.2.1. Dynamic Programming....5
3.2.2. Trellis (Graph)....6
3.3. Algorithm....7
3.3.1. Hart et al Algorithm....7
3.3.2. Parncutt et al Cost Calculation....9
4. Results....12
4.1. Suggested fingerings....12
4.2. Trellis Representation of the Fingerings....14
5. Discussion....16
5.1. Algorithm Suitability....16
5.2. Combining Hart Algorithm with Parncutt et al Costs....20
6. Conclusion....21
7. References....22
Appendix – Tables of Difficulty for Basic Movements....23

1. Introduction to Musical Terms

This is a basic introduction to the musical terms used in the report, which may be skipped over by those with musical vocabulary. A piano keyboard consists of several *octaves*, where each octave consists of twelve *semitones*. A note, in combination with possible modifier symbols, specifies the pitch of a corresponding tone (sound).

Each semitone corresponds to a key, and an octave (as shown in Figure 1) contains seven white keys and five black keys, where the first white key corresponds to the note C.

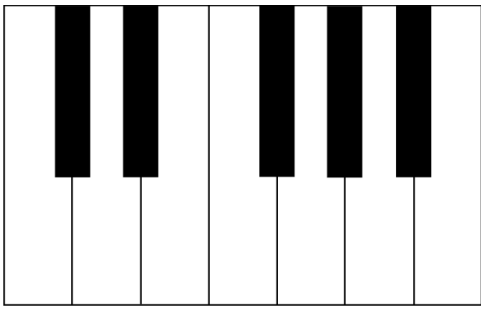

Figure 1: Example of an octave, the leftmost key corresponds to the C note.

The white keys in Figure 1 correspond to the notes C, D, E, F, G, A, and B, in order from left-to-right. In this report the black keys will be denoted by C#, D#, F#, G#, A#. The # symbol is chosen due to its similarity to the “sharp” symbol used in the notes (for examples see the notes in Section 4) and is used in a similar fashion.

The sharp symbol indicates a “raised” note, which on a piano means that the key immediately to the right of the key (that would have been played without a # symbol) will be used. The note C# for example indicates that the first black key in Figure 1 should be used rather than the first white key (C). Observe that in one of the melodies the note E# occurs, which denotes the same piano key as F, as F is the key immediately to the right of E. The reasons for this perceived redundancy will not be further addressed here.

Piano keys are ordered from left-to-right in order of increasing pitch. When referring to notes, a higher (higher-pitched) note is thus more to the right on the piano and a lower (lower-pitched) note is more to the left on the piano.

A pair of notes together make up an *interval*. The size of an interval is measured as the difference in seminotes between the notes. For example D-D# specifies a 1 note interval and C-E specifies a 4 note interval, since the sequence of semitones (keys) from C to E is C-C#-D-D#-E.

2.1. Background

There are several different schools of techniques for piano fingering, and all experienced piano players may have their own individual preferences. For this reason it is hard, if not impossible, to say what an optimal fingering for a specific piano piece would be. However there are many generally recognized requirements for any practically usable fingering, that stem from physiological limits. Automatically generated fingerings based on ergonomic criteria could be a valuable support for less experienced pianists.

Several studies are available on the subject of computer-based optimization of fingering for piano. One of the more famous studies on generating ergonomic piano fingering was published by Parncutt et al. They created a framework based on a system of twelve rules and four types of finger spans to control the generation of fingerings[1]. (For a short description of the rules used see section 3.3.2.)

Another group, Hart et al, created an algorithm for generating fingerings for the right hand, determined by a set of mathematical and experience-based assumptions which they implemented using dynamic programming[2]. (For a longer description of the paper see section 3.3.1.)

The set of possible fingerings may be represented using Trellis graphs[3, 4]. A description of Trellis graphs is provided in section 3.2.2. Kasimi et al introduced costs for gestures and movements that are measured by horizontal and vertical distance, where a gesture is a sequence of fingers to be pressed for a sequence of keys. A vertical cost is defined as the finger stretches within a gesture and a horizontal cost as the distance between a gesture and another. Zhu et al used trellis graphs to determine finger positions, and used constraints based on established piano techniques and gestures.

In summary, these papers on optimization of fingering for piano indicate that there are ways to design and adapt an algorithm, based on a set of rules, so that it generates one or more viable fingerings. There are several kinds of approaches for generating such fingerings, using graph theory or dynamic programming.

The purpose of this study was to gain further knowledge about how algorithms can be designed to generate practically usable piano fingerings automatically.

2.2. Problem Statement

The specific objective of the study was to design and implement an algorithm which automatically generates piano fingerings for one voice melodies played with the right hand, using a model with constraints based on ergonomic considerations and using dynamic programming for implementation. The algorithm was evaluated by comparing the result with fingerings based on established playing practice as suggested by experienced pianists.

3. Method

3.1. Approach

Consider a model that assigns a value to a given fingering based on the difficulty involved in playing the included sequences. Given that this model has a good enough set of rules, the fingerings with a low score (out of the total set of possible fingerings) should also be viable fingerings.

An algorithm based on such a rule-set needs to generate fingerings with the previous and following parts of the melody in mind, as fingers need to be in positions where they can quickly move to play coming notes. For short sequences it should be possible to find usable fingerings by doing a total search on the difficulty values for all the possible fingerings, but a better solution should be able to handle sequences of arbitrary length.

The approach was to select at least two melodies to be played with one hand, and implement an algorithm to generate fingerings for these pieces. Existing melodies were used and where necessary adapted to suit the needs.

The algorithm used MIDI-files of the selected melodies as input, and was initially supposed to give a string of numbers as output, where each number would correspond to a note and specify which finger should be used for playing that note (e.g. 1 for a note played using the thumb and 5 for the little finger). (However for reasons stated later in this report, this output approach was infeasible). The resulting fingerings were evaluated by comparing them to fingerings given by two professional pianists.

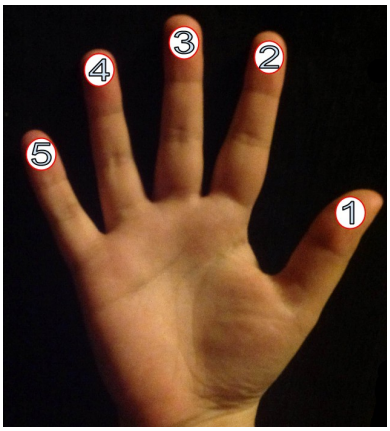

Fig. 2. Right hand with numbers used in piano fingering
1 = thumb, 2 = index finger, 3 = middle finger, 4 = ring finger, 5 = little finger

The algorithm reported by Hart et al [3] was considered to be a candidate for the given requirements. In every step of this algorithm the difficulty of playing the rest of the melody starting with a given finger is considered. Since this step is repeated for every note, both previous and coming notes of each specific note are taken into consideration in the final result. The algorithm is also time efficient since it has a linear time complexity in relation to the size of the input due to the dynamic programming methodology. In other words the algorithm requires an amount of time to generate fingerings that is linearly correlated with the number of notes in the melody. The paper describing the algorithm did not report any evaluation of the results except for a supplied 10-note long piece. For these reasons the choice for this study was to independently implement the original algorithm as well as a slightly expanded version and evaluate the algorithm by comparing the results with fingerings given by two pianists at professional level.

3.2. Mathematical Definitions

3.2.1. Dynamic Programming

Dynamic programming is a method for solving optimization problems that have *optimal substructure*. Such problems can be divided into smaller subproblems that each have an optimal structure. The results for these subproblems can be reused using *memoization*. Memoization helps to optimize by storing and reusing the results of the subproblems based on their given parameters. Dynamic programming solves each subproblem once and using memoization it may ignore subproblems that are never used. Compared to methods like depth-first search it usually has better time complexity.

Similar to the principle of induction (where a base case is expanded using an iterative rule) a dynamic programming solution often makes use of a base and then a recursive rule to expand subproblems (including the base case) into larger and larger subproblems until the original problem is solved.

3.2.2. Trellis (Graph)

A trellis is a form of acyclic graph where nodes can be grouped together in a way so that each group has an ordering in relation to other groups, each group representing a time slice from t_0 to t_n. Each node has at least one connection to another node in both immediately preceding and immediately following time slices. In the trellises used in this report, each node represents a finger and a time slice represents potential fingers for a certain note generated by the algorithm. Each potential fingering is connected to another fingering based on hand movements restricted by the set of rules.

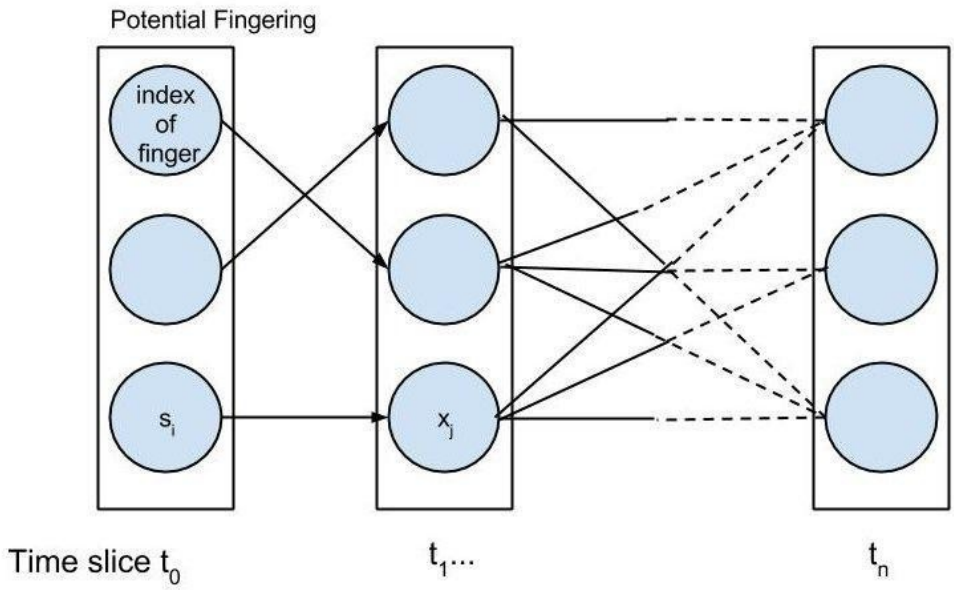

Fig. 3. Example of trellis graph representation

3.3. Algorithm

3.3.1. Hart et al Algorithm

In the algorithm [2] implemented in this study the problem it attempts to solve could be described as “find the minimum finger cost for a specified sequence of notes”. This problem has a base case, which is to “find the minimum finger cost for the last two notes, for fingers s and x”. The finger starting this last interval of two notes is denoted by *s* and the finger ending the interval is denoted by *x*.

To expand this algorithm using a dynamic programming approach, the problem can be rephrased as “find the minimum finger cost for the note sequence N_n = (n .. m), using finger pairs P = (s, x)”, where m is the final note and 0 < n < m. This time s is the finger playing note n and x the finger playing the note n+1. Notes are numbered using positive integers 1 to m. Using this version of the subproblem it is possible to go step by step from the problem of N_{m-1} = (m-1 .. m) to N_1 = (1 .. m). In each iteration the pairs P can be represented as a two-dimensional matrix of size 5 by 5, where element p_{sx} represents the minimum total cost of starting the current segment of the melody with finger s and ending it with finger x.

The algorithm uses a function $d_{i,j}(I_n)$ which gives the difficulty of playing the notes (n, n+1) using corresponding fingers (i, j). This difficulty is given by four lookup tables, shown in Appendix 1, with difficulty ratings on a scale between 1-4 based on an experienced piano player’s perceptions. Each table lists difficulty ratings for intervals of 1-12 semitones. Which table will be used is determined by whether the piano keys played are black or white. The black keys are shorter and extends above the white keys which has ergonomical implications.

To be able to handle one of the test melodies (*Für Elise*), which included a 13 semitone interval, a special case with a cost of 6 was added for all intervals of more than 12 semitones. This choice was made based on a premise that more than 12 semitones would require a hand movement for most, if not all people, if a piece is to be played ergonomically. The number 6 was chosen to give hand movement a notably higher difficulty rating than the other gestures.

For the last interval (m-1, m) the difficulty $f_n(i,j)$ of using fingers (i, j) is simply given by

$$d_{i,j}(I_{m-1})$$

as found in the difficulty lookup tables (see the Appendix). For all the previous intervals n the difficulty is given by

$$d_{i,j}(I_n) + f_{n+1}(j),$$

where $f_n(i)$ is the minimum cost

$$f_n(i) = min\{f_n(i,j)\}\ for\ j \in \{1..5\}$$

of playing from interval I_n until the end of the melody using finger j as the starting finger.

3.3.2. Parncutt et al Cost Calculation

The original algorithm used only the tables in the Appendix for cost calculation, but as this seemed too unspecific an alternative version was tried, where the cost was based on a subset of rules from the paper by Parncutt et al. This paper defines a set of twelve rules based on previous practice within the piano community. For each pair of fingers six different ratings are provided. They consist of maximum and minimum values for three types of limits of how far (or near) the fingers can reach; Prac (practical limit), Comf (comfortable limit) and Rel (relaxed limit). They are referred to as e.g. MinPrac for “minimum practical” limit and MaxRel for “maximum relaxed” limit. Thus the paper established limits for three different levels of ergonomic suitability, the optimal being “relaxed” and the least optimal being “practical” (practically possible).

Finger	MinPrac	MaxPrac	MinRel	MaxRel	MinComf	MaxComf
(1, 2)	-5	10	1	5	-3	8
(1, 3)	-4	12	3	7	-2	10
(1, 4)	-3	14	5	9	-1	12
(1, 5)	-1	15	7	10	1	13
(2, 3)	1	5	1	2	1	3
(2, 4)	1	7	3	4	1	5
(2, 5)	2	10	5	6	2	8
(3, 4)	1	4	1	2	1	2
(3, 5)	1	7	3	4	1	5
(4, 5)	1	5	1	2	1	3

Table 1. Table of the six different ratings based on the rating table from Parncutt[1, p 346]. The first column shows the index of fingers, the first number is the first note pressed, the second the next note.

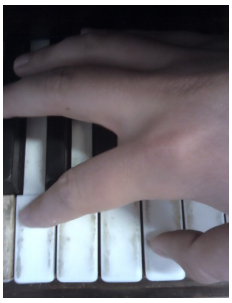
Fig. 4. MinPrac for 1-2

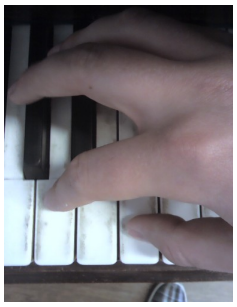
Fig. 5. MinComf for 1-2

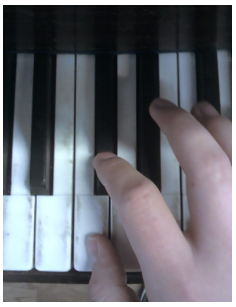
Fig. 6. MinRel for 1-2

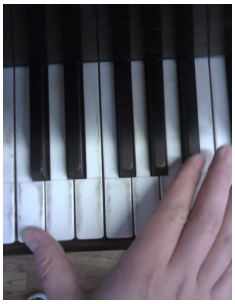
Fig. 7. MaxPrac for 1-2

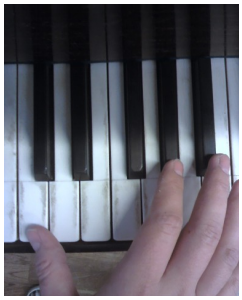
Fig. 8. MaxComf for 1-2

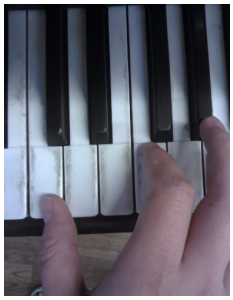
Fig. 9. MaxRel for 1-2

Figures 4 through 9 show an example of the limits for the finger pair 1-2 (thumb - index finger). As seen in the pictures, negative values in the table are for when fingers cross each other.

The rules are briefly described as follows:

1. Stretch Rule: Assign 2 points for each semitone that an interval exceeds MaxComf or is less than MinComf. This rule gives a penalty for practically possible but uncomfortable movements.

2. Small-Span Rule: Assign 2 points for each semitone that an interval is less than MinRel if the thumb is not used, or 1 point for each semitone if the thumb is used. This rule assigns a penalty for small span intervals, for fingerings, that while possible, and perhaps even comfortable, are not relaxed. The rule takes into consideration that the thumb may be moved more freely than other fingers.

3. Large-Span Rule: Assign 1 point for each semitone that an interval exceeds MaxRel for finger pairs including the thumb, or 2 points for finger pairs not including the thumb. This rule is equivalent to rule 2 but for intervals with a large span.

6. Weak-Finger Rule: Assign 1 point for every time finger 4 or finger 5 is used. This rule assigns a penalty when the ring finger or little finger is used, since these fingers are considered to be weaker than the others.

8. Three-to-Four Rule: Assign 1 point for every time finger 3 is immediately followed by finger 4. This rule assigns a penalty when the middle finger and ring fingers are used in sequence, since the movement of these fingers is interdependent.

9. Four-on-Black Rule: Assign 1 point for every time fingers 3 and 4 occur consecutively in any order with 3 on white and 4 on black. This rule assigns a

penalty to the middle finger-ring finger sequence when moving from a white key to a black key, since the black keys are shorter and higher and thus provide even more difficulty to an already problematic finger combination (see rule 8).

10. Thumb-on-Black Rule: Assign 1 point for every time the thumb plays a black key, additional 2 points if it is immediately preceded by a white key and additional 2 points if it is immediately followed by a white key. This rule assigns a penalty for playing a black key with the thumb, and even more if preceded or followed by white keys. Since the thumb is shorter than other fingers it is harder for it to play the shorter and higher black keys, and general hand movement becomes even more difficult if the longer fingers play white keys before or after.

12. Thumb-Passing Rule: Assign 1 point for each thumb- or finger-pass that is made between notes of the same color. Assign 3 points if the higher note is black, played by the thumb and the lower note is white, played by a finger other than the thumb. This note assigns a penalty to thumb- or finger-passes, i.e. when fingers cross each other. For reasons similar to those given for rule 10, thumb-on-black and other finger on white is an especially problematic combination, which is made even more difficult by a thumb-pass.

Note that rules 4, 5, 7 and 11 could not be implemented within the scope of this paper, and are excluded from this list. All four of the excluded rules require three sequential notes to be known to calculate the cost, which is not possible within the original structure of the Hart et al algorithm.

4. Results

4.1. Suggested fingerings

The algorithm was used to generate fingerings for three melodies with different characteristics. The music scores of the melodies are shown in Appendix 2.

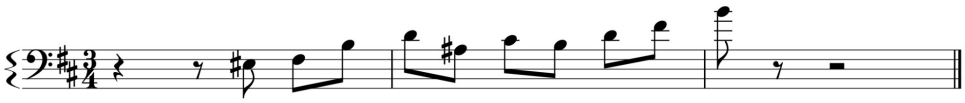

Fig 10: Right-hand passage from Chopin's first Scherzo

Melody 1
Right-hand passage from *Scherzo No. 1* by Chopin, shown in figure 10. Selected because it was used as an example in the report by Hart et al.

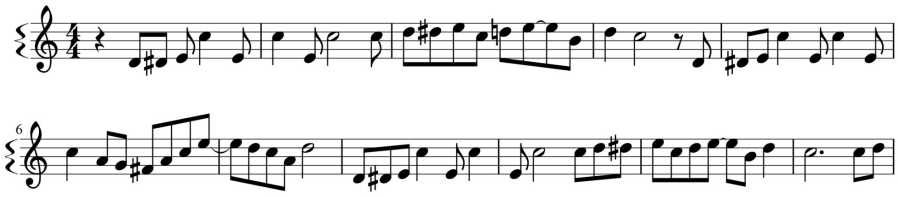

Fig. 11: Beginning of The Entertainer

Melody 2
Beginning of *The Entertainer* by Scott Joplin as shown in figure 11, adapted for playing with the right hand.

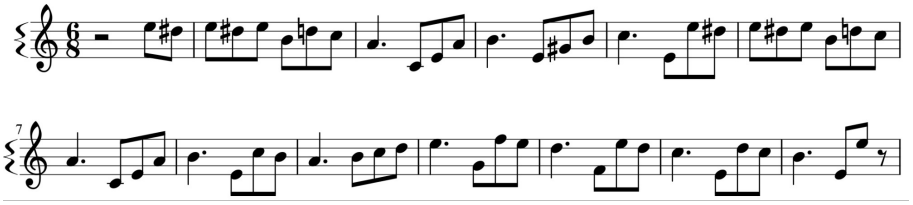

Fig. 12: Beginning of Für Elise

Melody 3
Beginning of *Für Elise* by Beethoven, adapted for playing with the right hand.

For melodies 2 and 3 the fingerings given by the pianists had some differences, indicated by the blue markings. Table 2 to 4 contains the suggested fingerings for melody 1 to melody 3 from the pianists.

Melody 1		
Pianist 1	Pianist 2	Note
1	1	E#
2	2	F#
4	4	B
5	5	D
2	2	A#
3	3	C#
1	1	B
2	2	D
3	3	F#
5	5	B

Melody 2		
Pianist 1	Pianist 2	Note
1	1	D
2	2	D#
1	1	E
5	5	C
1	1	E
5	5	C
1	1	E
5	5	C
1	1	C
2	2	D
3	3	D#
4	4	E
1	1	C
3	3	D
4	4	E
1	1	B
4	3	D
3	2	C
1	1	D
2	2	D#
1	1	E
5	5	C
1	1	E
5	5	C
1	1	E
5	5	C
4	3	A
3	2	G
2	1	F#
1	2	A
3	3	C
5	5	E
4	4	D
3	3	C
2	2	A
5	4	D
1	1	D
2	2	D#
1	1	E
5	5	C
1	1	E
5	5	C
1	1	E
5	5	C
1	1	C
2	2	D
3	3	D#
4	4	E
1	1	C
3	3	D
4	4	E
1	1	B
4	3	D

Melody 3		
Pianist 1	Pianist 2	Note
5	5	E
4	4	D#
5	5	E
4	4	D
5	5	E
2	2	B#
4	4	D
3	3	C
2	2	A
1	1	C
2	2	E
4	4	A
5	5	B
1	1	E
3	2	G#
4	3	B
5	4	C
1	1	E
5	5	E
4	4	D#
5	5	E
4	4	D
5	5	E
2	2	B#
4	4	D
3	3	C
2	2	A
1	1	C
2	2	E
4	4	A
5	5	B
1	1	E
5	5	C
4	4	B
1	1	A
2	2	B
3	3	C
4	4	D
5	5	E
1	1	G
5	5	F
4	4	E
3	3	D
1	1	F
5	5	E
4	4	D
3	3	C
1	1	E
5	5	D
4	4	C
3	3	B
1	1	E
5	5	E

Table 2. Suggested fingerings for Scherzo No.1.
Table 3. Suggested fingerings for The Entertainer.
Table 4. Suggested fingerings for Für Elise.

4.2. Trellis Representation of the Fingerings

Figures 13-15 show trellis graphs of the fingerings given by the algorithm for all melodies. Paths coinciding with the fingerings suggested by the pianists are colored in blue. Pianist fingerings missing entirely in the algorithm result graphs are either colored in red, or marked by red boxes.

The melody from Chopin's Scherzo no. 1 only had two different fingerings given by the algorithm, one of which matched both expert fingerings.

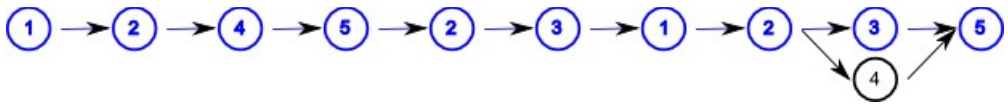

Fig. 13. *Trellis showing the results for Melody 1 (Passage from* Chopin's Scherzo no. 1*).*

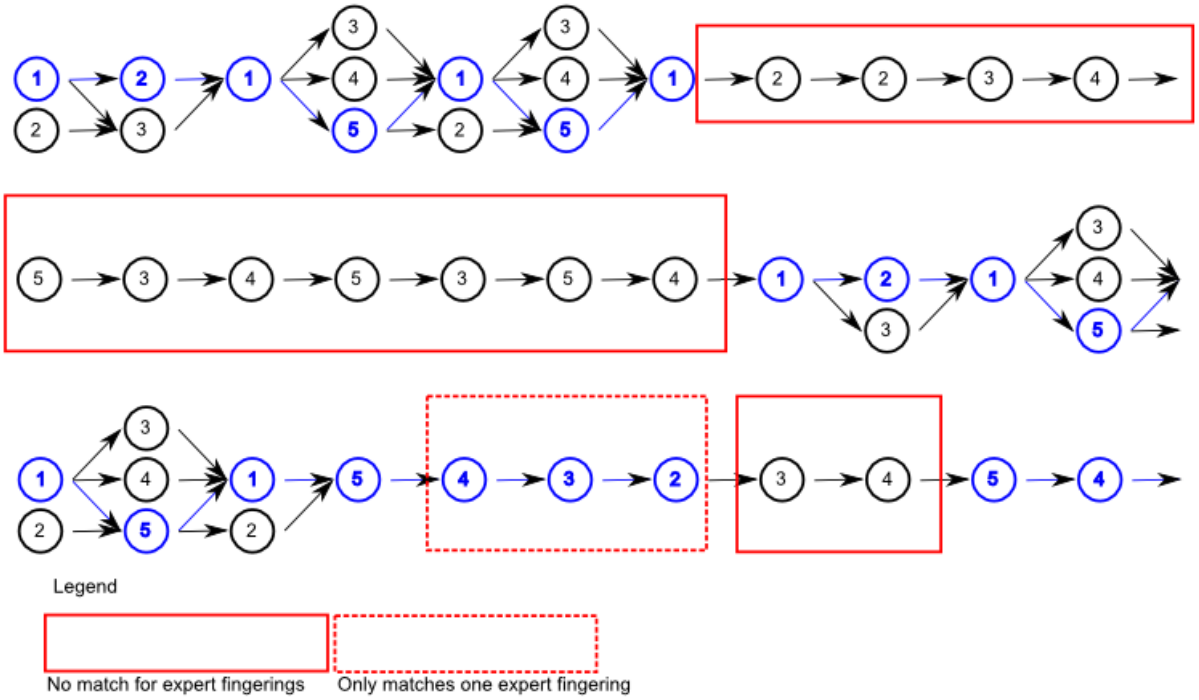

Fig. 14. *Trellis showing the results for Melody 2* (The Entertainer) *excerpt.*

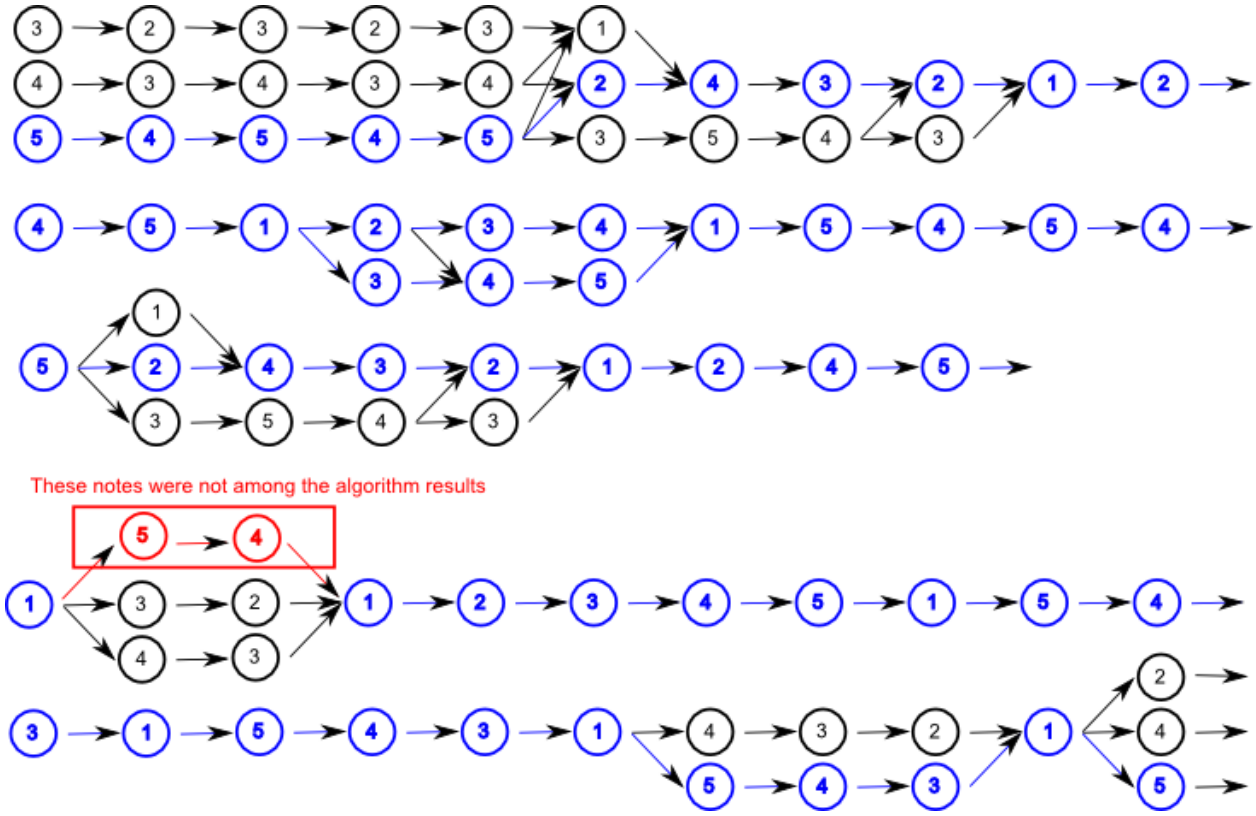

Fig. 15. *Trellis graph of results for Melody 3* (Für Elise) *excerpt*.

5. Discussion

5.1. Algorithm Suitability

Out of the fingerings for three melodies evaluated with the algorithm and the professional pianists, one melody (Melody 1: Passage from *Chopin, Scherzo no. 1*) showed consistent results. For the other two melodies, the algorithm generated many permutations of possible fingerings (see trellis graphs in Section 4.2). This result indicates that the algorithm had good enough basic performance but requires further evaluation and development.

Figures 13 to 15 show trellises of the fingerings that the algorithm considered to be optimal (i.e. had minimal cost). Every possible path in these graphs going from left to right was assigned the same total cost. Each of the possible paths in the given graphs is thus considered to be equally optimal by the difficulty model used by the algorithm. Because of this the difficulty model seems to be too unspecific when considering different fingerings. This is most likely related to the limited range of difficulties given in the lookup tables. Since only values 1-4 are used to specify the difficulties there is little or no discrepancy between neighboring intervals and neighboring fingering alternatives in large parts of the data.

For Melody 1 both pianists ended with the finger sequence 2-3-5, but the algorithm also allowed the sequence 2-4-5. A walkthrough of the cost calculation will show why. The three notes involved are D-F#-B. D-F# is an interval of four semitones. The lower note is played by a white key and the higher note is played by a black key, and the interval is thus assigned difficulty from column 4 (corresponding to the interval) of lookup table 2 (corresponding to the white-black combination) in the appendix. This gives difficulty 2 for finger sequence 2-3 (finger 2 -- the index finger -- plays lower note, finger 3 -- the middle finger -- plays higher note) and difficulty 1 for finger sequence 2-4 (index finger plays lower note, ring finger plays higher note). The interval F#-B is an interval of five semitones, with lower key black and upper key white. The difficulty is thus given by Table 3 in the appendix. Column 5 of Table 3 gives difficulty 1 for finger sequence 3-5 (middle finger plays lower note, little finger plays higher note) and difficulty 2 for finger sequence 4-5 (ring finger plays lower note, little finger plays higher note). The 2-4-5 sub-sequence thus ends up with difficulty 2+1= 3 and 2-3-5 with 1+2 = 3.

Fig. 16, 17. Playing D-F#-B using 2-4-5 sequence

Fig. 18, 19. Playing D-F#-B using 2-3-5 sequence

These movements are shown in Figure 16-19. The 2-4 (Fig. 16) and 3-5 (Fig. 19) movements (both of which were assigned difficulty 1) seemed somewhat equivalent in strain on the hand, but the 4-5 movement caused a noticeably higher strain on the hand than the 2-3 movement, which is consistent with the fact that both pianists chose 2-3-5 instead of 2-4-5. Speculating on exact numbers seems infeasible as this would depend on the entirety of the tables, but it seems that given a larger range of difficulty values, the 4-5 movement should have a higher cost for interval 5 than 2-3 has for interval 4, with a black-white key sequence. Given more time it is likely that similar cases would be discovered in the majority of the tables, giving a stricter definition of which fingerings are considered optimal and improving the results.

The Trellis graphs further reinforce the fundamental problem mentioned earlier, when using the algorithm for the current purposes. Since the algorithm rates a large set of different fingering sequences as equally difficult, it becomes highly impractical to generate strings of all the fingerings generated by the algorithm, even when restricting the generation to the best-rated fingerings. Considering the

result from *The Entertainer* alone (See Figure 14), the graph contains on the order of $2^6 \times 3^4 = 5184$ different fingering combinations only considering the first five bars, and a great deal more if the whole melody is considered.

For *The Entertainer*, the results of the algorithm did not include the fingerings suggested by the pianists other than in minor sections. An initial evaluation of the fingerings given by the algorithm shows that they are playable without any major issues, but the fingerings given by the pianists are easier to play.

The main reason for the discrepancy between the algorithm results and pianist given fingerings in this case is most likely that both pianists chose to use movements that are undefined by the Hart et al algorithm. The algorithm only includes intervals 1-12, but at several positions in *The Entertainer* the same note is repeated twice (i.e. a 'zero interval' between notes). The implementation used the same finger for the two notes, but that seems to have been a poor decision, since both pianists used the opportunity to switch fingers between the notes in these situations to put the hand in a better position for coming notes.

An alternative would be to give a zero cost to all finger combinations when the same note is played twice, but this does not seem to match reality, as there are different difficulties involved in playing the same note using e.g. fingers 3-4 (middle finger, ring finger) versus fingers 1-2 (thumb, index finger). A suggestion for future research would be to add a column for the zero interval to each difficulty table, using input from one or more experienced pianists.

For Melody 3 *Für Elise* (see Fig. 15) in contrast, the trellis graphs include both of the pianist fingerings except for two notes, where the fingerings (4-3) and (3-2) are given by the algorithm as alternatives. These fingerings are similar in style though not exact agreement with the (5-4) fingering given by both pianists for this interval. A similar discrepancy occurs in the beginning of the melody, where both 5-4-5-4-5 (chosen by both pianists), 4-3-4-3-4 and 3-2-3-2-3 are given as alternatives. For this first set of notes the fingering selected by the pianists is motivated by the soft dynamic level (pianissimo), a factor which is not considered by the algorithm.

The results could be considered as a good starting point, but it would be preferable if the algorithm could be more selective, giving separate ratings for the top choices instead of assigning the same difficulty to a large set of alternatives. A more selective algorithm could be evaluated more properly to determine whether it ranks individual fingerings in a way that an expert pianist does.

The range of difficulty values (1-4) in the lookup tables from the Hart et al study (see appendix) is likely too small to be able to differentiate properly between viable and less viable fingerings. Further, they were based only on the personal preferences of a single pianist. It is likely that other pianists would have different

opinions on the difficulty values used in the tables, and even from a layman's perspective some of the values seem questionable. One example is that playing an interval of 11 semitones (one semitone less than a full octave) using the thumb and index finger is only rated as difficulty level 2, which seems questionable. Another highly questionable result of the range and values used is that the movement 5-1 (i.e. playing the lower pitch note with the little finger and the higher pitched note with the thumb, as shown in Figure 20) is rated equally difficult for all intervals as the movement 3-1 (middle finger playing lower note, thumb playing higher, shown in Figure 21).

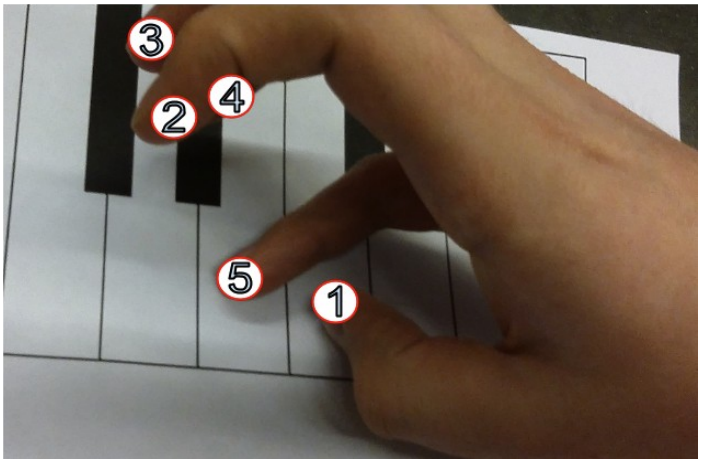

Fig. 20: Playing an interval using fingers 5-1

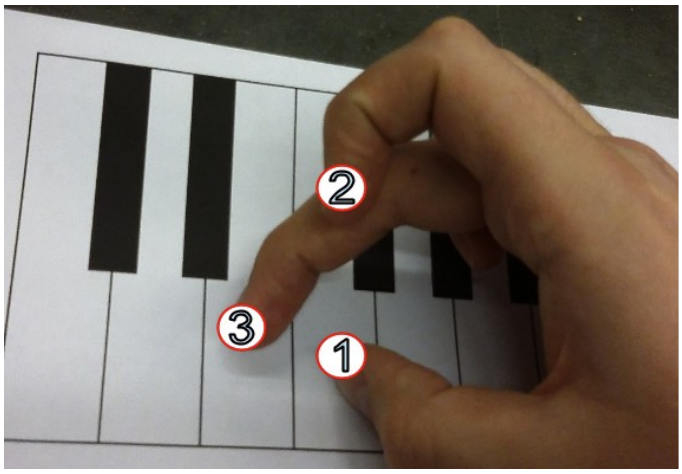

Fig. 21: Playing an interval using fingers 3-1

The former fingering seems practically impossible for all intervals, while the latter is possible though quite impractical at least for intervals of about three semitones or less. A revision of the tables with difficulty values using input from more professional pianists would be highly desirable to improve the performance of the algorithm.

5.2. Combining Hart Algorithm with Parncutt et al Costs

The implemented algorithm would probably give better results if it also used rules similar to those in Parncutt et al [1], instead of only using predetermined difficulty values given by the four lookup tables in the appendix. Such an approach would give a wider range of ratings and thus likely help in differentiating between fingering alternatives.

Since the cost calculation of the Hart et al algorithm could be considered a weak point, an attempt was made of replacing the cost calculation for intervals with one along the lines of Parncutt et al, while keeping the other parts of the algorithm the same. Parncutt et al described twelve rules (see section 3.3.2), and as, four of the rules (rules 4, 5, 7 and 11) could not be implemented because they required knowledge of three notes in sequence. The Hart et al algorithm has knowledge of two notes only at each step, and increasing this to three would require a larger redesign of the algorithm, which is not within the scope of this study.

As expected, it was possible to decrease the amount of solutions generated by the algorithm drastically by using this more precise rule set. With the subset of rules that was possible to implement, the algorithm generated only a single fingering for *The Entertainer* and $2^6 \times 3 = 192$ fingerings for the entire *Für Elise* excerpt. However, the fingerings generated by this combined Hart-Parncutt algorithm were not of the same high standard as the fingerings presented above. This was probably related to the incomplete set of rules that was used.

6. Conclusion

The implemented algorithm generated fingerings that to a considerable extent were consistent with those proposed by the pianists, but also a large superset of less optimal fingerings. For one melody, (Melody 1: *Chopin - Scherzo no.1*), the algorithm generated the same fingering as suggested by the pianists. For Melody 3 (*Für Elise*) it generated most of the proposed fingerings but for Melody 2 (*The Entertainer*) it was not very successful, probably because of the repeated notes (zero size interval) used in the melody at multiple times.

The results indicate that there are promising possibilities for generating viable fingerings for piano automatically. Several models are applicable, such as Parncutt et al or Hart et al. Both these models give frameworks for generating useful fingerings. However the approach proposed by Parncutt et al seems like a better basis for future work since it has a set of rules that is more specific, has been evaluated by a larger group of professionals and is based on a greater body of knowledge.

7. References

[1] Parncutt R., Sloboda J., Clarke E., Raekallio M. and Desain P. “An ergonomic model of keyboard fingering for melodic fragments,” *Music Perception*, Vol.14, pp. 341 - 382 (1997).

[2] Hart M., Bosch R., and Tsai E. “Finding optimal piano fingerings” The Undergraduate Mathematics and Its Applications Journal, UMAP, 2000, pp. 1-10.

[3] Zhu Y.F., Ramakrishnan A.S., Hamann B., Neff M. “A system for automatic information of piano performance, ”*Computer Animation and Virtual Worlds, DOI: 10.1002/cav.1477 (2012)*

[4] Kasimi, A., Nichols, E., & Raphael, C. (2007). A Simple Algorithm for Automatic Generation of Polyphonic Piano Fingerings. Proc of the International Conference on Music Information Retrieval (ISMIR 2007), Vienna, Austria.

Appendix – Tables of Difficulty for Basic Movements

The following tables of difficulty ratings of fingering of two consecutive notes are based on the tables by Hart et al[1]. It is founded on one experienced pianist's considerations with the rating system going from 1 to 4, where 4 indicates the most difficult movements. Each table is based on ratings for intervals between 2 and 12 semitone steps and the color of the first and second key. The letter x indicates impossible movements. To be able to handle Melody 2 (*Für Elise*), which included a 13 semitone interval, a special case with a cost of 6 for all intervals larger than 12 semitones was added. (see Section 3.3.1)

The tables are defined by lower note and higher note, independent of which order the notes are played. For example, the finger pair (1, 2) means that the thumb plays the lower note (the leftmost key) and the index finger plays the higher note (the rightmost key).

The tables are ordered as follows:
Table 1: Both fingers play white keys
Table 2: Lower key is white, higher key is black
Table 3: Lower key is black, higher key is white
Table 4: Both fingers play black keys

Table 1

Fingering (lower, upper)	1	2	3	4	5	6	7	8	9	10	11	12
(1, 2)	1	1	1	1	1	1	1	2	2	2	2	3
(1, 3)	1	1	1	1	1	1	1	1	1	2	2	3
(1, 4)	2	2	1	1	1	1	1	1	1	1	1	2
(1, 5)	3	3	2	2	1	1	1	1	1	1	1	1
(2, 1)	2	2	3	3	4	4	4	4	4	4	4	4
(2, 3)	1	1	1	1	2	2	3	3	3	3	3	3
(2, 4)	2	2	1	1	1	1	2	3	3	3	3	3
(2, 5)	3	3	2	2	1	1	1	1	1	2	2	2
(3, 1)	2	2	3	3	4	4	4	4	4	4	4	4
(3, 4)	1	1	2	2	3	3	3	3	3	3	3	3
(3, 5)	3	3	1	1	1	1	3	3	3	3	3	3
(4, 1)	2	2	4	4	4	4	4	4	4	4	4	4
(4, 5)	1	1	1	1	3	3	3	3	3	3	3	3
(5, 1)	4	4	4	4	4	4	4	4	4	4	4	4

1 Hart M., Bosch R., and Tsai E. “Finding optimal piano fingerings” The Undergraduate Mathematics and Its Applications Journal, UMAP, 2000, pp 3-4.

Table 2

Fingering (lower, upper)	1	2	3	4	5	6	7	8	9	10	11	12
(1, 2)	1	1	1	1	1	1	1	1	2	2	3	x
(1, 3)	1	1	1	1	1	1	1	1	1	1	2	x
(1, 4)	2	2	2	1	1	1	1	1	1	1	1	x
(1, 5)	3	3	3	2	2	2	1	1	1	1	1	x
(2, 1)	4	4	4	4	4	4	4	4	4	4	4	x
(2, 3)	1	1	1	2	2	3	3	3	3	3	3	x
(2, 4)	2	1	1	1	1	2	2	2	3	3	3	x
(2, 5)	3	2	2	2	2	1	1	1	2	2	3	x
(3, 1)	4	4	4	4	4	4	4	4	4	4	4	x
(3, 4)	1	1	1	3	3	3	3	3	3	3	3	x
(3, 5)	3	2	2	2	2	2	3	3	3	3	3	x
(4, 1)	4	4	4	4	4	4	4	4	4	4	4	x
(4, 5)	2	2	2	2	3	3	3	3	3	3	3	x
(5, 1)	4	4	4	4	4	4	4	4	4	4	4	x

Table 3

Fingering (lower, upper)	1	2	3	4	5	6	7	8	9	10	11	12
(1, 2)	3	2	2	1	1	2	2	2	3	3	3	x
(1, 3)	3	2	2	1	1	1	2	2	2	2	3	x
(1, 4)	3	3	3	1	1	1	1	1	2	2	2	x
(1, 5)	3	3	3	2	2	2	1	1	1	1	1	x
(2, 1)	2	3	3	4	4	4	4	4	4	4	4	x
(2, 3)	1	1	1	2	2	3	3	3	3	3	3	x
(2, 4)	2	1	1	1	1	2	3	3	3	3	3	x
(2, 5)	3	2	2	1	1	1	1	1	1	1	2	x
(3, 1)	2	3	3	4	4	4	4	4	4	4	4	x
(3, 4)	1	1	1	3	3	3	3	3	3	3	3	x
(3, 5)	2	1	1	1	1	1	2	2	3	3	3	x
(4, 1)	3	4	4	4	4	4	4	4	4	4	4	x
(4, 5)	1	1	1	2	2	3	3	3	3	3	3	x
(5, 1)	3	4	4	4	4	4	4	4	4	4	4	x

Table 4

Fingering (lower, upper)	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
(1, 2)	x	2	2	2	2	x	3	3	3	3	x	3
(1, 3)	x	2	2	2	2	x	2	2	2	2	x	2
(1, 4)	x	3	2	2	2	x	2	1	1	1	x	2
(1, 5)	x	3	3	3	3	x	2	1	1	1	x	1
(2, 1)	x	2	3	3	4	x	4	4	4	4	x	4
(2, 3)	x	1	1	1	2	x	3	3	3	3	x	3
(2, 4)	x	2	1	1	1	x	2	3	3	3	x	3
(2, 5)	x	3	2	2	1	x	1	1	1	2	x	2
(3, 1)	x	3	4	4	4	x	4	4	4	4	x	4
(3, 4)	x	1	1	1	2	x	3	3	3	3	x	3
(3, 5)	x	3	1	1	2	x	3	3	3	3	x	3
(4, 1)	x	4	4	4	4	x	4	4	4	4	x	4
(4, 5)	x	2	2	2	3	x	3	3	3	3	x	3
(5, 1)	x	4	4	4	4	x	4	4	4	4	x	4